

CIS 4930/CIS 6930 (Spring 2025)
Hardware Accelerators for Machine Learning
Dept. of Computer Science & Engineering
University of South Florida

Title

Road Sign Recognition

Authors

Manjal Amrishkumar Patel (U20390166)

Pearl Viralkumar Patel (U67099453)

Manan Hareshbhai Patel (U31049615)

Instructor

Dr. Dayane A. Reis

Date

8th May 2025

Abstract

Road Sign Recognition is a machine learning project. It classifies traffic signs from images. The project uses both a CPU and a GPU to compare how fast and well they perform. We developed a custom CNN model to recognize 43 classes of road signs. We used the German Traffic Sign Recognition Benchmark (GTSRB) dataset, which has over 50,000 images. We used data preprocessing and data augmentation. This helped the model to learn better and generalize well. First, we built a basic CNN model and tested its performance on both the CPU and the GPU. Next, we improved our method by making two special CNN models. One was a lightweight model for the CPU. The other was a deeper and more complex model for the GPU. We used the Adam optimizer with learning rate scheduling and regularization techniques (dropout, batch normalization) to refine training. The final GPU-accelerated model achieved 97.5% test accuracy, outperforming the CPU-based model's 91.9% accuracy, while also delivering approximately 22× higher throughput in inference. This demonstrates significant performance gains from hardware acceleration.

Overview

Traffic sign recognition is important for driver assistance systems and self-driving cars. It helps to detect road signs in real time for safety and navigation. The GTSRB dataset has over 50,000 images of German traffic signs in 43 categories. It includes signs like Speed Limit (70 km/h), Stop, No Entry, Slippery Road, Road Work, Turn Right Ahead and many more. These images have different lighting, obstacles, and angle changes, which makes the task challenging. It helps the model to learn to recognize traffic signs in real-world conditions, not just in perfect images. This improves accuracy and reliability in real driving situations. The goal of this project is to design a machine learning model that can accurately classify road sign images into their correct classes, achieve over 95% accuracy using proper data augmentation and preprocessing, and compare the performance of CPU and GPU by running the model on both. We achieved this by developing a convolutional neural network from scratch and deploying it on CPU and GPU. We used 80% of the data for training, and from that, 20% was used as a validation set. The remaining 20% of the data was used as a test set.

Bibliographic Study

To develop a base from existing research, we reviewed six key papers related to deep learning for image recognition, traffic sign classification, and hardware acceleration.

1. **(Krizhevsky, Sutskever et al. 2012)** - ImageNet Classification with Deep CNNs: Krizhevsky and colleagues designed a convolutional neural network to recognize objects in pictures. This system was tested on a vast collection of images, and it was able to sort

them into many distinct groups. The authors use a deep CNN architecture with five convolutional layers, three fully connected layers, and a final softmax classifier. They use GPU acceleration for efficient training. To enhance performance and avoid overfitting, they introduce techniques such as ReLU (Rectified Linear Unit) activation, dropout regularization, and data augmentation. Their method gives the best results, performing much better than the earlier methods. However, the approach is computationally intensive, requiring multiple GPUs and days of training, making it less accessible for those without high-performance hardware. Additionally, the fixed image size may make it harder for the model to work well with pictures of different sizes. This work demonstrated that deep CNNs can achieve state-of-the-art accuracy given sufficient data and computation, motivating our use of deep CNN architecture and GPU training for road sign recognition.

2. **(Stallkamp, Schlipsing et al. 2011)** - GTSRB: A Competition and Dataset for Traffic Sign Recognition: The authors address the problem of traffic sign recognition by creating the German Traffic Sign Recognition Benchmark (GTSRB) which is a large-scale dataset consisting of 50,000+ images belonging to 43 traffic sign classes. This dataset comes from collected real-world video footage that aims to capture variation in lighting, variability in weather, occlusion, and rotation. To ensure fair performance evaluation, they hold a multi-class classification competition on which contestants implement Convolutional Neural Networks (CNNs), Support Vector Machines (SVMs), and Linear Discriminant Analysis (LDA). Notably, convolutional neural networks outperformed other methods, establishing CNNs as a promising approach for traffic sign classification. However, the authors observed challenges such as difficulty in distinguishing visually similar signs (e.g. speed limits) at low resolutions. They emphasized the need for larger and more diverse training data and suggested incorporating detection pipelines for better real-world applicability. This study provided the primary dataset for our project and reinforced the use of CNNs, with the GTSRB serving as our training and evaluation benchmark.
3. **(Simonyan and Zisserman 2014)** – Very Deep CNNs (VGG) for Image Recognition: Simonyan and Zisserman analyzed the impact of network depth on image classification performance. They used small (3 x 3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16-19 weight layers. Their results showed that increasing depth significantly improves accuracy on large-scale image recognition tasks. They also found that certain design choices like smaller receptive fields and the omission of Local Response Normalization (LRN) led to better performance and reduced memory usage. The authors demonstrate that the representation depth is beneficial for the classification accuracy, and that state-of-the-art performance on the ImageNet challenge dataset can be achieved using conventional ConvNet architecture with increased depth. This research helped our project to consider a deeper model (on the GPU) for higher accuracy, while being aware of memory and

computation constraints on hardware. We adopted concepts like stacking many convolutional layers and using only small filters, as evidenced by VGG, to improve representation learning in our road sign CNN.

4. **(Khan, Park et al. 2023)** - Lightweight CNN for Traffic Sign Recognition: Khan and colleagues proposed a compact CNN architecture for traffic sign classification that minimizes computational cost while maintaining high accuracy. They overcome the overfitting issues that GoogleNet, VGG16, and ResNet had, resulting in less computationally expensive models. Their model was trained on GTSRB and another dataset (BelgiumTSC), using extensive data augmentation (rescaling, normalization, flips) and hyperparameter tuning (grid search for optimizers and learning rates). The network employed convolutional, max-pooling, batch normalization, and global average pooling layers. With the Adam optimizer and categorical cross-entropy loss, their lightweight CNN achieved 98.41% accuracy on GTSRB (and 92.06% on the BelgiumTS dataset). Yet, it struggles with occlusions, lighting changes, and unequal class distribution, and even its use of data augmentation might hinder its real-life application. This paper validated that a smaller CNN can be effective for traffic signs, guiding our design of a simplified model for the CPU.
5. **(Chen, Liu et al. 2024)** - Hardware Acceleration of Traffic Sign Recognition (FPGA & Spiking Neural Network): Chen and colleagues present the challenges of recognizing traffic signs in self-driving cars. Usually, this is done using Convolutional Neural Networks, but they use too much computational power and work slowly. As a solution to these limitations, the authors introduce an FPGA-based Spiking Convolutional Neural Network (SCNN) system, called SFPGA-TSRS. This system includes a method called spatial attention (SA) with Leaky Integrate-and-Fire (LIF) neurons to help the model to focus on important parts of the image. Their approach uses Spiking Neural Networks (SNNs) because they save energy. It also improves accuracy and computational speed by hardware optimizations. While the results of the research show that the proposed system reduces power consumption and enhances recognition speed on the GTSRB dataset, but still, it has some challenges which include complexity of implementing deep SNN architectures and processing in real-time can be difficult because SNNs need multiple steps to work. Additionally, it is not clear if this method will work well for larger datasets or more complex tasks. While we did not implement spiking networks, the focus on energy efficiency and throughput informed our performance evaluation metrics (e.g., measuring operations per second and considering energy usage).
6. **(Glavtchev, Muyan-Özçelik et al. 2011)** – GPU-based Feature Detection for Speed Limit Signs: The authors explore how a graphics processing unit (GPU) can help to detect speed-limit signs in real time. Their approach uses a feature-based radial symmetry detector

(RSD). It is implemented with Compute Unified Device Architecture (CUDA) and OpenGL. The GPU's ability to handle many tasks at once helps to make detection fast. They chose this method to achieve real-time performance on hardware with limited capacity. It helps to overcome the slow processing of CPU-based systems. Their system uses grayscale video input from a camera on a moving vehicle. It detects speed-limit signs with high accuracy and runs smoothly on a simple computer with an Intel Atom CPU and an NVIDIA GeForce GPU. However, their method has some limitations. It uses only grayscale images, which may not work well in low-contrast situations. The system is designed for speed-limit signs, so it may not detect other traffic signs or work well in complex environments. This paper highlighted that GPUs can dramatically accelerate image processing algorithms.

Description of Methods

Dataset and Preprocessing: We used the GTSRB dataset, which provides color images of traffic signs at varying resolutions (15×15 up to 250×250 pixels). We first resized all images to a fixed 32×32-pixel resolution (maintaining 3 color channels) as input to our CNN. The pixel values were normalized (scaled) to a [0,1] range and we subtracted the training set mean from each channel (a common normalization step). The data was then split into training, validation, and test sets: we reserved 20% of the training data for validation during training and used the official GTSRB test set for final evaluation. This split was done randomly.

Data Augmentation: To improve generalization, training samples were augmented with a random rotation (up to ± 5 degrees), random translation ($\pm 10\%$ shift horizontally and vertically), and random zoom/scaling ($\pm 10\%$ zoom). These augmentation parameters were chosen to mimic real variations in sign appearance (rotation on poles, slight scale changes due to distance, etc.) while keeping the sign recognizable. Only the training images undergo augmentation; validation and test images remain unaltered to accurately measure performance. The augmentation was performed offline for both GPU and CPU. Figure 1 illustrates an example of our augmentation: the original image is a speed limit sign, and the augmented version shows a rotated and slightly translated instance of the same sign.

Model Architecture (CPU vs GPU): We designed two CNN architectures: a smaller model for CPU execution and a deeper model for GPU. This approach was taken because the GPU can handle a larger network within reasonable time, whereas the CPU benefits from a simpler network to keep inference time manageable. The CPU model consists of 3 convolutional layers, each followed by a ReLU activation; the first two convolutional layers are followed by a 2×2 max-pooling layer to down sample spatial dimensions. We included one dropout layer (after the second pooling) to reduce overfitting. After flattening, the network has one fully connected (dense) layer with 256 neurons (ReLU) and a final output dense layer with 43 neurons (softmax) for classification. In

total, this CPU model has about 321,323 parameters. This architecture was our “baseline” and is relatively lightweight, inspired by the network used by Khan et al. for efficiency on limited hardware.

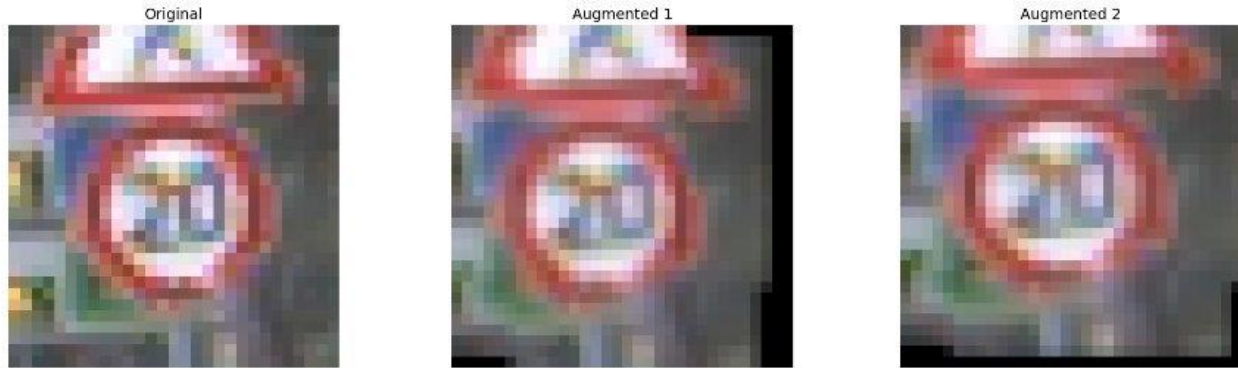


Figure 1: The first image is the original image and the two images that follow are augmented images after random rotations, translations, and scaling.

For the GPU model, we increased the depth and capacity of CNN. The GPU model has 4 convolutional layers (with more filters per layer than the CPU model), arranged in two blocks: each block has two conv layers followed by a max-pooling layer. We also added batch normalization after each convolutional layer to stabilize and speed up (this was part of our method update after initial experiments). Dropout layers (with dropout rate 0.3) are placed after each pooling and after the first dense layer to further reduce overfitting. The GPU model uses 2 fully connected layers with higher dimensions (we used 512 units for the first dense vs. 256 in the CPU model) before the output layer. All convolutional layers use ReLU activations, and the final output uses softmax. This deeper network has 18,401,195 parameters. While this is $\sim 3\times$ the parameter counts of the CPU model, it remains good enough to train on the GPU quickly. Both models use cross-entropy loss for training since it’s a multi-class classification problem.

Training Strategy: We trained the models using the **Adam optimizer** (which provides efficient stochastic gradient descent with adaptive learning rates). Based on initial tuning, we set the learning rate to 0.0008 for both models. Additionally, we employed a **learning rate scheduler** that reduces the learning rate by half every 10 epochs (StepLR) to allow finer convergence in later training. Training was conducted for 30 epochs on the GPU model and 50 epochs on the CPU model. Training on the GPU was much faster (in the order of minutes per epoch) than on the CPU, owing to the GPU’s parallelism. During training, we tracked the training and validation loss and accuracy of each epoch to ensure the model was learning effectively. We saved the model state that achieved the best validation accuracy and used that for final testing.

Progress

We approached the project in iterative milestones as guided by the course instructor:

- **Milestone 1 (Literature Review):** We surveyed relevant research papers to inform us of our model design. This gave us direction to use deeper CNNs and data augmentation, and to leverage GPU acceleration for training. We also became familiar with the GTSRB dataset characteristics from the literature, which guided our preprocessing.
- **Milestone 2 (Initial Methods Description):** We proposed our initial CNN architecture and implementation plan. At this stage, we intended to use the same CNN architecture for both CPU and GPU, consisting of 3 convolutional layers and 2 dense layers, anticipating over 95% accuracy. We planned different augmentation strategies for each device to cope with hardware constraint. We set up the development environment on Google Colab and by the end of Milestone 2, we had a preliminary training script, and the baseline model structure coded.
- **Milestone 3 (Updated Methods & Initial Results):** Based on initial experiments, we updated our approach. We noticed the single-architecture approach left the CPU training very slow and the model underperforming on validation. Thus, we used the separate CPU and GPU models as described, adding batch normalization and increasing the GPU model's depth. Initially, after training ~30 epochs the GPU model reached about 96% validation accuracy, whereas the CPU model for ~30 epochs stopped at 89% validation accuracy. These results indicated that the CPU model needed either more depth or more training. Given CPU limitations, we decided to train it for more epochs rather than increase its size. We also had not yet integrated learning rate scheduling in Milestone 2; by Milestone 3 we added it, which helped the models continue improving after the initial epochs. In the meantime, we are still figuring out how to measure the energy consumption of the CPU model, for GPU model we were planning to use NVIDIA performance management library to measure the GPU performance metric. Milestone 3 concluded with partial results and a plan to push further.
- **Final Implementation and Testing:** We trained the CPU model for 50 epochs with the updated configuration, allowing it to reach over 90% accuracy. We also fine-tuned the GPU model (e.g., slightly adjusting dropout rates and augmentation magnitude) and obtained a final test accuracy above 97%. We conducted multiple test runs to gather statistics on runtime, throughput, and energy using NVIDIA's pynvml library for GPU power metric and Python's psutil for CPU usage. However, psutil does not give the exact measure of the CPU energy consumption, it gave the approximate value by simulation. Hence, comparison in terms of energy consumption for CPU and GPU is not done. By

project end, we had met our accuracy target on the GPU and documented a clear performance benefit of using the GPU over the CPU.

Results

Model Accuracy: Our GPU model achieved a final test accuracy of 97.51% on the test set, exceeding the 95% goal. The CPU model achieved a test accuracy of 91.90%, falling slightly short of 95% but still demonstrated strong performance for a lightweight model.

Training Curves: Figure 2 illustrates the GPU training curves, which indicate a very rapid convergence. Both training and validation accuracy cross 97% within the first 10 epochs and remain stable throughout the rest of training. The loss curves decrease sharply in the initial epochs and plateau near zero, indicating that the model not only learned efficiently but also avoided significant overfitting. This behavior suggests that the deeper GPU model, coupled with regularization techniques such as dropout and batch normalization, effectively generalize.

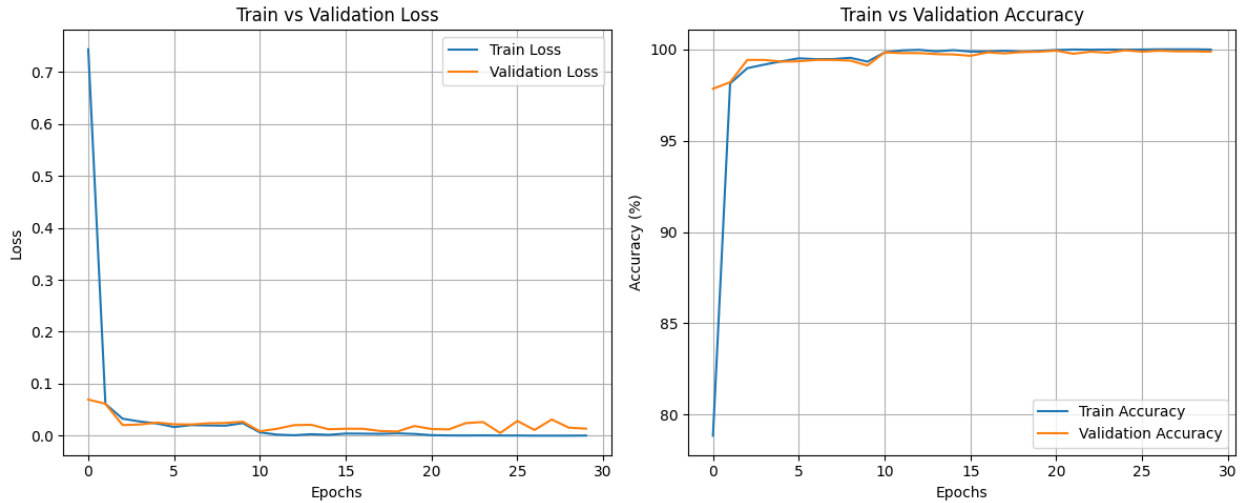


Figure 2: Training and validation accuracy (right), loss (left). The curves are nearly overlapping, demonstrating excellent generalization.

In contrast, the CPU model, trained for more epochs, shows a more gradual increase in accuracy. The validation accuracy stabilizes just under 95% by the end of training, while training accuracy is near 90%, indicating a possible underfitting due to model capacity limits. The wider gap between training and validation losses suggests that the CPU model had to work harder to generalize across the more challenging examples in the GTSRB dataset. Figure 3 shows the CPU training curves.

Performance Metrics (CPU vs GPU): We measured inference runtime, throughput (Giga-operations per second), and estimated energy consumption for each. We also computed energy efficiency and power consumption on GPU. For each metric an average of 5 test runs are

considered. The model MACs for CPU and GPU are 8485803 and 473439531, respectively. Table 1 and 2 depict the results of GPU and CPU performance evaluation, respectively.

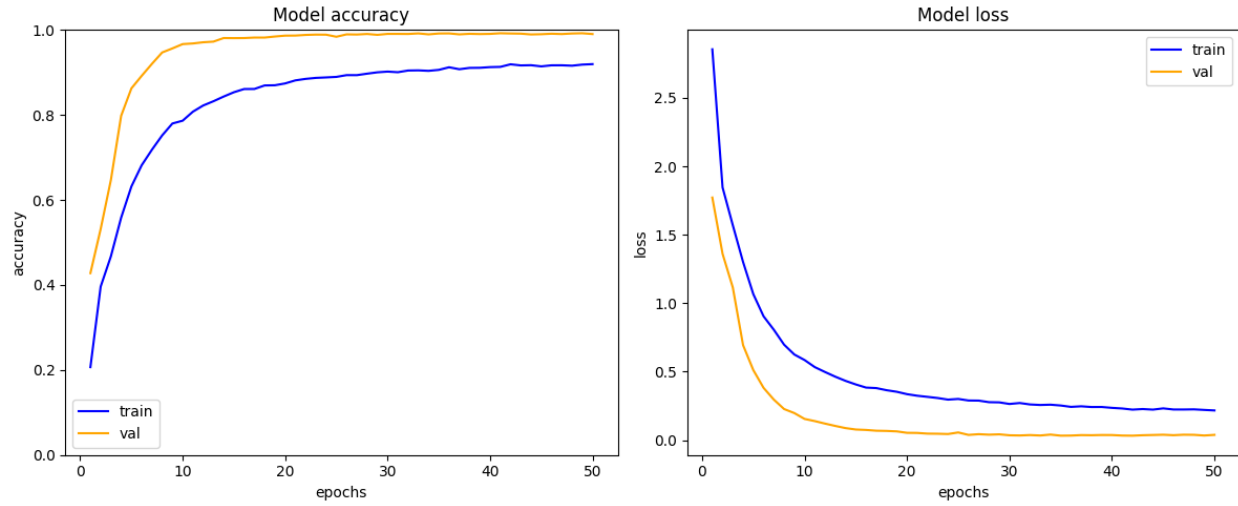


Figure 3: Training and validation accuracy (left), loss (right). Slower convergence and a persistent gap indicate limited capacity.

Tests	Runtime (s)	Throughput (GOPS)	Energy Efficiency (GOPS/W)	Energy (J)	Power (W)
Test 1	3.58	24.38	0.11	222.48	31.9
Test 2	3.67	23.81	0.10	245.51	32.3
Test 3	3.77	23.25	0.10	231.95	34.4
Test 4	4.69	18.60	0.07	278.89	38.7
Test 5	3.89	22.45	0.16	138.20	32.1
Average	3.92	22.49	0.108	223.40	33.88

Table 1: Performance Metrics for GPU (TESLA T4)

Tests	Runtime (s)	Throughput (GOPS)	Energy (proxy units)
Test 1	3.03	1.03	0.40
Test 2	3.06	1.02	0.30
Test 3	3.16	0.99	0.46
Test 4	2.95	1.06	0.38
Test 5	2.98	1.05	0.39
Average	3.03	1.03	0.38

Table 2: Performance Metrics for CPU (APPLE M4)

The GPU achieved an average throughput of 22.49 GOPS, significantly outperforming the CPU's 1.03 GOPS, demonstrating approximately $22\times$ faster processing capability. This validates the GPU's parallelism and its ability to handle large matrix operations efficiently. Interestingly, the GPU runtime (3.92 s) appears longer than the CPU runtime (3.03 s). However, this is due to the higher number of operations in the GPU model. Therefore, although the GPU is executing a much more complex model, it completes it within a comparable timeframe—highlighting the GPU's computational advantage. Additionally, energy efficiency for the GPU averaged 0.108 GOPS/W, with an average energy consumption of 223.40 J. These results demonstrate that, despite higher power drawing, the GPU is more energy-efficient when scaled to real-time applications.

The CPU model was intentionally lightweight to maintain feasibility for sequential execution. Even with fewer MACs, the CPU struggles to match the GPU's throughput due to lack of parallelism. While the CPU appears to complete inference faster in wall-clock time, this is misleading—its model is roughly $56\times$ less complex than GPU model.

Analysis: The superior accuracy of GPU model is due to its higher capacity. The CPU model, with fewer parameters to maintain speed, could not quite reach the same accuracy. However, 91.9% is still respectable accuracy and might be improved with further techniques. The experiment shows that the hardware can decide the feasible complexity of model. With a GPU, we were able to train a large model that generalize better, whereas on CPU we favored lighter model due to computational constraints.

Key Takeaways and Conclusion

This project showed that a custom CNN model, along with good preprocessing and data augmentation, can recognize road signs with high accuracy using the GTSRB dataset. Our final models gave better results than expected. The GPU model reached 97.51% accuracy, and the CPU model reached 91.90% accuracy. One of the major takeaways is the significant performance gap between CPU and GPU execution. The GPU model provides about 22 times more throughput than the CPU model. This shows that hardware accelerators make training and testing faster and allow us to use deeper and more complex models. Data augmentation methods like rotation, translation, and scaling helped the model to generalize better and avoid overfitting. We used these methods only during training to keep the evaluation results consistent. We saw that it is important to match the model design with the hardware. Lightweight CNNs work better on CPUs with limited parallelism. Deeper CNNs work better on GPUs, giving higher accuracy and faster results.

In conclusion, choosing the right model design for the hardware, along with good data augmentation and training methods, can make deep learning models more accurate and faster for tasks like traffic sign recognition. This project highlights the important role of hardware accelerators such as GPU, in real world machine learning applications.

Acknowledgement

We would like to express our gratitude to Dr. Dayane A. Reis for her guidance and supervision throughout this project. We also thank Parsa Khorrami for his valuable assistance and technical support. This work was completed as part of the CIS 4930/6930 Hardware Accelerators for ML course at the University of South Florida (USF). Their insights and feedback were instrumental in completing this work.

References

Chen, H., et al. (2024). "Research on Hardware Acceleration of Traffic Sign Recognition Based on Spiking Neural Network and FPGA Platform." IEEE Transactions on Very Large Scale Integration (VLSI) Systems.

Glavtchev, V., et al. (2011). Feature-based speed limit sign detection using a graphics processing unit. 2011 IEEE Intelligent Vehicles Symposium (IV), IEEE.

Khan, M. A., et al. (2023). "A lightweight convolutional neural network (CNN) architecture for traffic sign recognition in urban road networks." Electronics **12**(8): 1802.

Krizhevsky, A., et al. (2012). "Imagenet classification with deep convolutional neural networks." Advances in neural information processing systems **25**.

Simonyan, K. and A. Zisserman (2014). "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556.

Stallkamp, J., et al. (2011). The German traffic sign recognition benchmark: a multi-class classification competition. The 2011 international joint conference on neural networks, IEEE.